

Web Applications

React JS

Suheil Hammoud

■ Quick Start

- How to create and nest components
- How to add markup and styles
- How to display data
- How to render conditions and lists
- How to respond to events and update the screen
- How to share data between components

■ Creating and Nesting Components

React apps are made out of components.

A component is a piece of the UI that has its own logic and appearance.

```
function MyButton() {  
  return (  
    <button>I'm a button</button>  
  );  
}
```

React component names must start with a capital letter.

You can nest a component into another:

```
export default function MyApp() {  
  return (  
    <div>  
      <h1>Welcome to my app</h1>  
      <MyButton />  
    </div>  
  );  
}
```

■ Writing Markup with JSX

JSX is a syntax extension for JavaScript, used to describe what the UI should look like.

```
function AboutPage() {  
  return (  
    <>  
      <h1>About</h1>  
      <p>Hello there.<br />How do you  
do?</p>  
    </>  
  );  
}
```

- JSX is stricter than HTML
- Use a parent wrapper
- Always close tags

■ Adding Styles

Use `className` instead of `class` in JSX.

```
<img className="avatar" />
```

Define the style in CSS:

```
.avatar {  
  border-radius: 50%;  
}
```

React doesn't prescribe how you include CSS - use a `<link>` tag or a build tool.

■ Displaying Data

JSX lets you embed variables and expressions using `{}`.

```
<h1>{user.name}</h1>
<img
  className="avatar"
  src={user.imageUrl}
/>
```

■ Displaying Data

Complex example:

```
const user = {  
  name: 'Hedy Lamarr',  
  imageUrl: 'images/img.png',  
  imageSize: 90,  
};
```

```
export default function Profile() {  
  return (  
    <>  
      <h1>{user.name}</h1>  
      <img  
        className="avatar"  
        src={user.imageUrl}  
        alt={'Photo of ' + user.name}  
        style={{  
          width: user.imageSize,  
          height: user.imageSize  
        }}  
      />  
    </>  
  );  
}
```

Conditional Rendering

Use regular JS conditions or ternary operators in JSX.

```
let content;  
if (isLoggedIn) {  
  content = <AdminPanel />;  
} else {  
  content = <LoginForm />;  
}  
return <div>{content}</div>;
```


Conditional Rendering

Compact version:

```
<div>
  {isLoggedIn ? <AdminPanel /> : <LoginForm />}
</div>
```

Logical &&:

```
<div>
  {isLoggedIn && <AdminPanel />}
</div>
```

■ Rendering Lists

Use `map()` to turn an array into JSX elements.

```
const products = [
  { title: 'Cabbage', id: 1 },
  { title: 'Garlic', id: 2 },
  { title: 'Apple', id: 3 },
];

const listItems = products.map(product =>
  <li key={product.id}>{product.title}</li>
);

return <ul>{listItems}</ul>;
```

Keys help React identify list items and optimize updates.

Conditional List Styling

Use logic in props to style list items.

```
const products = [
  { title: 'Cabbage', isFruit: false, id: 1 },
  { title: 'Garlic', isFruit: false, id: 2 },
  { title: 'Apple', isFruit: true, id: 3 },
];

export default function ShoppingList() {
  const listItems = products.map(product =>
    <li
      key={product.id}
      style={{color: product.isFruit ? 'magenta' : 'darkgreen'}}
    >
      {product.title}
    </li>
  );

  return <ul>{listItems}</ul>;
}
```

■ Responding to Events

Declare event handlers inside components.

```
function MyButton() {  
  function handleClick() {  
    alert('You clicked me!');  
  }  
  
  return (  
    <button onClick={handleClick}> Click me </button>  
  );  
}
```

Note: Don't call the function in JSX (`onClick={handleClick}` not `onClick={handleClick()}`).

■ Updating the Screen with State

Use the `useState` hook to “remember” information.

```
import { useState } from 'react';

function MyButton() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }
}
```

```
return (
  <button onClick={handleClick}>
    Clicked {count} times
  </button>
);
}
```

Each instance of the component gets its own state.

■ Multiple Independent States

```
import { useState } from 'react';

export default function MyApp() {
  return (
    <div>
      <h1>Counters that update separately</h1>
      <MyButton />
      <MyButton />
    </div>
  );
}
```

Each `<MyButton />` maintains its own independent `count`.

■ Using Hooks

- Hooks are functions starting with `use`
- `useState` is a built-in hook
- Hooks must be called at the top level of components (not inside loops or conditions)
- You can define your own hooks too

Sharing Data Between Components

To synchronize state between components, lift the state up to their common parent.

```
import { useState } from 'react';

function MyButton({ count, onClick })
{
  return (
    <button onClick={onClick}>
      Clicked {count} times
    </button>
  );
}

export default function MyApp() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
  }
}
```

```
return (
  <div>
    <h1>Updated together</h1>
    <MyButton
      count={count}
      onClick={handleClick}
    />
    <MyButton
      count={count}
      onClick={handleClick}
    />
  </div>
);
```

Props let you pass data and handlers from parent to child.

Summery

You've learned:

- Component creation
- JSX syntax
- Styling
- Displaying data
- Handling events
- Managing state
- Lifting state

References:

- **React Docs** (<https://react.dev/learn>)